AI Engineer Assignment: Teacher AI Platform

1. Objective

Design and implement an AI-powered system that converts raw educational documents into a structured, classroom-ready **Teacher Knowledge Package (TKP)**.

Note for Candidates: We are evaluating your ability to design modular AI architectures, orchestrate complex workflows, implement educational reasoning, and produce production-ready engineering. This is *not* a simple prompt-engineering task; it is an AI system design challenge.

---

2. Problem Statement

Teachers often possess high-quality educational resources (research papers, textbook chapters, lecture notes, PDFs, PPTs). However, these resources are not immediately teachable.

Your task is to build an AI system that transforms such documents into classroom-ready teaching material. The system must understand the content, organize it into pedagogically meaningful structures, generate teaching artifacts, and package everything into a reusable format.

---

3. The Assignment (What You Need to Do)

You are required to build a pipeline that processes an uploaded document through the following 10 stages.

Phase 1: Document Intelligence & Knowledge Extraction

- **Stage 1: Document Intelligence:** Parse the uploaded document (PDF, DOCX, PPT, or Text). Extract text while preserving structure (detect headings, sections, tables, figures, equations, and metadata).
- **Stage 2: Educational Classification:** Determine the document's context. Generate metadata including Subject, Grade, Difficulty, Topic, Chapter, Category, and Language.
- **Stage 3: Knowledge Extraction:** Build a structured educational representation. Extract Learning Objectives, Prerequisites, Concepts, Definitions, Formulae, Keywords, Examples, Applications, and Common Misconceptions.

Phase 2: Pedagogical Planning & Generation

- **Stage 4: Teaching Planner:** Convert the extracted knowledge into a multi-period teaching strategy (e.g., splitting a chapter into 5, 40-minute periods, complete with learning objectives and sequencing).
- **Stage 5: Classroom Content Generation:** Generate detailed teaching material for *every* period. This includes Entry Tickets (warm-ups), Teacher Scripts, Blackboard Notes, Classroom Activities, Checkpoint Questions, Exit Tickets, Homework, and a "Mentor Moment" (motivational anecdote).
- **Stage 6: Activity Generation:** Design diverse activities (Demonstrations, Role Play, Experiments, etc.) complete with duration, materials needed, teacher instructions, and success criteria.
- **Stage 7: Assessment Generation:** Create comprehensive assessments (MCQs, Short/Long answers, Numerical problems) along with answer keys and rubrics.
- **Stage 8: Learning Gap Analysis:** Identify potential student misconceptions. Provide diagnostic questions, severity levels, and remedial actions for each gap.

Phase 3: Validation, Output, and Orchestration

- **Stage 9: Validation:** Implement an automated validation step to check for JSON schema adherence, hallucination detection, missing objectives/concepts, and consistency across periods.
- **Stage 10: Publishing:** Package the final output into a master `TeacherKnowledgePackage.json`, alongside easily consumable formats and a simple UI to easily evaluate the gen content (e.g., PDFs for Lesson Plans, Teacher Guides, Assessment Books).
- **Streaming Progress API:** Implement a mechanism to stream progress updates for the long-running AI jobs to the frontend (e.g., `{"stage": "lesson-generation", "progress": 60}`).

---

4. Suggested Architecture Reference

While you are free to design the exact implementation, your system should logically follow a microservice or pipeline-based architecture resembling the following:

`API Gateway` ➔ `Upload Service` ➔ `Document Intelligence` ➔ `Educational Classification` ➔ `Knowledge Extraction` ➔ `Teaching Planner` ➔ `Content/Activity/Assessment Generators` ➔ `Validation Engine` ➔ `Database/Storage`.

---

5. Submission Requirements (What We Need From You)

To successfully complete this assignment, you must submit the following:

1. **A Deployed, Working Prototype (MANDATORY):** A live URL where we can upload a document and view the generated Teacher Knowledge Package. (You may use Streamlit, Vercel, Render, AWS, etc., for hosting).
2. **Source Code:** A link to a public/private Git repository containing all backend, AI orchestration, and frontend code.
3. **System Documentation (README):**
   - Setup instructions to run the project locally.
   - A high-level Architecture Diagram.
   - Explanation of the AI orchestration framework or patterns used (e.g., LangChain, LlamaIndex, custom agents).
4. **Sample Outputs:** Include at least 2 sample `TeacherKnowledgePackage.json` files generated by your system in a `/samples` folder.

---

6. Evaluation Criteria

Your submission will be evaluated holistically based on the following weights.

Critical Output Quality Note: *A major part of the evaluation will assess how well your system handles a diverse range of subjects (e.g., STEM topics with equations vs. Humanities texts with subjective narratives) and varying levels of topic complexity.*

Category	**Weight**	**What We Evaluate**
Content Generation & Versatility	25%	Lesson quality, activities, assessments, structured outputs, and **adaptability/robustness across different subjects and topic ranges.**
Educational Understanding	20%	Topic classification, learning objectives, and concept extraction accuracy regardless of domain.
Teaching Planning	20%	Multi-period sequencing, pedagogical flow, adaptation to class duration.

Category	Weight	What We Evaluate
Document Intelligence	15%	Parsing accuracy, structure preservation, metadata extraction.
Engineering & Architecture	15%	Modular design, orchestration, API design, progress streaming, validation.
Documentation & Demo	5%	Code quality, README clarity, architecture diagram, and prototype usability.

---

7. Bonus Features (Stand out from the crowd)

Candidates may earn additional credit for implementing any of the following:

- **Multi-Agent Orchestration:** Clear separation of responsibilities among different AI agents.
- **RAG & Traceability:** Retrieval-Augmented Generation over uploaded docs with citation/source traceability.
- **Curriculum Alignment:** Aligning outputs with specific boards (e.g., Common Core, CBSE, ICSE).
- **Performance Optimization:** Batching, caching, parallel generation, and cost management.
- **Observability:** Logs, metrics, tracing, and automated retry mechanisms for AI calls.
- **Multilingual Support:** Ability to generate lesson plans in different languages.